Start here if you want the kit to make sense

GODSHAND Starter Manual

The practical reader's guide to the cinematic AI film kit

The clearest entry point into the GODSHAND ecosystem: architecture, creative inputs, demo logic, packaging, and beginner-friendly setup material gathered into one book.

Curated from original source PDFs with layouts preserved.

Commercial edition. This starter manual is aimed at readers or buyers who want the clearest, most approachable orientation to the GODSHAND workflow without every engineering appendix.

Imprint: LifeDocs Press
Source archive: lifedocs.zip
Included source documents: 5
Editorial method: sequence curation, front matter, contents, section divider pages, bookmarks, and metadata.
Preservation policy: original source PDF pages are inserted without rewriting their internal text or layouts.

Contents

5 source documents

1. Build Map and Source of Truth 4

2. Creator Guide 12

3. Demo Film: The Beholder's Truce 20

4. Kit Listing and Setup Sheet 24

5. Project Tutorials 34

Build Map and Source of Truth

The canonical orientation document and conceptual foundation of the kit.

The following pages reproduce the original source PDF as part of this curated collection.

🧭

GODSHAND Kit — Build Map & Source of Truth

🟠 **Correction to my earlier note on this page — you were right.** My first pass said "every file is byte-identical, nothing was lost." That was wrong, and I am fixing the record. The core rendering **engine** (all of `src/`, every workflow, `run.py`, the configs) *is* byte-for-byte identical in every build — that part is safe. **But the builds are NOT all equal:** two builds dropped whole features, and one build's launcher was silently **regressed** to an inferior version. Your instinct that "we lost more than the folder" was correct.

✅ What is safe (identical in every build)

The actual engine never changed. These are byte-for-byte identical across FINAL_, TUNED_AK, and FINAL_AK:

- All of `src/` (`render_motion.py`, `forge_character.py`, `voice_pass.py`, `score_gen.py`, `stitch.py`, `postprocess.sh`, …)
- All `workflows/*.json`, `run.py`, and every `config_*.yaml` / `shots_*.json`
- All character reference images in `refs/<name>/face` and `/body`

Nothing about how your film actually renders was corrupted. The damage is only in the **launchers, docs, and scaffolding** that wrap the engine.

⚠️ Feature audit — what was DROPPED or OVERRIDDEN (25 Jul 2026)

Compared file-by-file by content hash, with packaging nesting normalized.

Feature	FINAL_ (your Downloads)	TUNED_AK	FINAL_AK
:--	:-:	:-:	:-:
`NEW_STORY_TEMPLATE/` (author any new film)	✅	❌ dropped	❌ dropped
`GODSHAND_ONECLICK.sh` (main launcher)	✅ full (10,565 B)	❌ dropped	⚠️ **regressed** (9,120 B)
`README_L4.md`	✅ full (3,437 B)	❌ dropped	⚠️ **regressed** (3,129 B)
`l4_model_setup.py` (WAN 2.2 downloader)	✅	❌ dropped	✅
`notebooks/` (2 Kaggle notebooks)	✅	❌ dropped	✅
`RUN_TUNED.sh` (one-command tuned launcher)	❌	✅ only here	❌

🚨 The important one: the launcher was REGRESSED in FINAL_AK

`GODSHAND_ONECLICK.sh` exists in two versions. **The good one is in your Downloads (FINAL_).** The FINAL_AK copy is 1,445 bytes smaller and was rewritten in a way that reintroduces the exact "wrong folder" behavior you have been fighting:

Good version (FINAL_ / Downloads):

- Story is **selectable** — `KIT_CONFIG` picks any film (not just the confrontation).
- Characters are read **dynamically** from the config's `character_forge.targets`.
- Character photos are **OPTIONAL** — if `kit/refs/<name>/face` has images it uses them, otherwise it **auto-forges** a consistent face, logging the choice per character.
- `GODSHAND_ANCHORS` images are copied into `kit/refs/` (the correct home).
- Explicitly states: *"There is no `kit/anchors/` folder — the real location is `kit/refs/<name>/`."*

Regressed version (FINAL_AK):

- **Hardcoded** to the confrontation film + "26 shots" — loses the any-story engine.
- Drops the per-character auto-forge detection.
- Reintroduces a phantom `kit/anchors/` folder and dumps anchor images into `ComfyUI/input/` and the **kit root** — this is the stray-image / wrong-location behavior that caused the Spider-Man / Joker contamination.
- Its `README_L4.md` tells you to use the non-existent `kit/anchors/`.

➡️ **Use the launcher and README from FINAL_ (Downloads). Do not use the FINAL_AK versions.**

✅ Which build to use (corrected)

`GODSHAND_FINAL_` **in your Downloads is your MASTER build.** It has the correct launcher, `NEW_STORY_TEMPLATE/`, the notebooks, `l4_model_setup.py`, and the full README — everything, in its correct form.

The only thing it lacks is `RUN_TUNED.sh` (one-command tuned launcher), and you do not strictly need it — its tuned values are already baked into `NEW_STORY_TEMPLATE/config_newstory.yaml` (`denoise: 0.55`, `codeformer_fidelity: 0.5`, `active_profile: lightning_l4`). To also get the one-command launcher, copy it in:

```
cp GODSHAND_TUNED_AK/RUN_TUNED.sh GODSHAND_FINAL/kit/
```

> Correction: I earlier told you to use `TUNED_AK` as the "richest" build. That was wrong — TUNED_AK actually dropped the launcher, `l4_model_setup.py`, the notebooks, and `NEW_STORY_TEMPLATE/`. FINAL_ (Downloads) is the right master.

🕷️ Root cause — why Spider-Man / Joker appeared as “jack” (the original question)

Three things had to line up. All three are now confirmed in the code:

1. **Two stray images were sitting in jack’s staging folder.**
   `refs/jack/_inbox/jack_preface.jpg` is a Spider-Man (Tobey Maguire) still; `refs/jack/_inbox/joker.png` is a Joker still. `_inbox/` is a “dump a pile of candidate

photos here" staging area that `cull_photos.py` is meant to sort into `face/` and `body/` — but it was never run on jack, so the strays stayed.

2. **The reference loader reads recursively.** `comfy_client.py` and `render_motion.py` collect refs with `glob("refs/jack/**/*", recursive=True)` — so they reach **into `_inbox/`** instead of reading only `refs/jack/face/`.
3. **`_inbox` sorts first, and only the first 5 refs are used.** An underscore sorts before letters, so the recursive list starts with the two `_inbox` images. The code keeps `refs[:5]`, so **Spider-Man and Joker became jack's top-two identity anchors**, ahead of his real face.

And the safety net failed too: the identity-check gate (`_identity_score`) uses the *same* recursive glob, so it scored rendered frames against Spider-Man as if that were jack — it did not just miss the drift, it **approved** it.

Net effect: the blueprint still was conditioned on Spider-Man/Joker → jack animated as Spider-Man → the QC gate rubber-stamped it.

The fix

- **Immediate (done):** `GODSHAND_MASTER.zip` has every `refs/*/_inbox/` folder deleted, so the recursive glob can only find real face/body images.
- **Permanent (one-line, recommended):** restrict the ref globs to the face/body folders, never the parent. In `comfy_client.py`, `render_motion.py` (`_identity_score`), and `refs_for()`, read `refs/<char>/face/*` (and `/body/*`) instead of `refs/<char>/**/*`. Then even if junk lands in `_inbox` again, it can never reach a render.

📁 Where the key things live (never search blind again)

- **Main launcher (correct version):** `GODSHAND_ONECLICK.sh` — in FINAL_ / your Downloads
- **Tuned launcher:** `RUN_TUNED.sh` — TUNED_AK only
- **Model download:** `l4_model_setup.py`
- **Author a new film:** `NEW_STORY_TEMPLATE/` (two files: a shots JSON + a config YAML)

- **Confrontation story + render config:** `config_confrontation.yaml` + `shots_confrontation.json`
- **Character images (real):** `refs/<name>/face/` and `/body/` — cast: `jack, julie, kunal, kunal_40s, kunal_young, gunja, observer, original_identity`
- **The correct anchor location is `kit/refs/<name>/` — there is NO `kit/anchors/` folder.**
- **Junk to delete:** `refs/<name>/_inbox/` — the stray Spider-Man / Joker / Scarlett placeholders live here. The clean design never reads from here.

🎛️ Tuning baked into `RUN_TUNED.sh`

> kaggle_2xt4 profile (2× Tesla T4 — Kaggle has NO L4) · `KIT_MOTION_DENOISE=0.55` (motion freedom, was 0.25) · `KIT_FLF2V_ENDFRAME=1` (shot-to-shot morph ON) · `KIT_FACE_ENHANCE=0` (skip plasticky face composite) · softened temporal denoise · codeformer 0.9 → 0.5

Renders 4 proof shots: `confr_caught`, `confr_allblades`, `confr_dare`, `confr_dontouch`.

🔒 On the sandbox (why your originals were never at risk)

I only ever held a read-only, throwaway copy of the kits you uploaded. I never had write access to your Kaggle kit, your datasets, or your machine. Any differences above already existed in the builds you sent me — I did not create them.

🗒️ Change Log (newest first)

- **25 Jul 2026, ~7:57 AM IST — Fixed the recurring L4 mistake — the proof render now targets Kaggle 2× Tesla T4, not L4 (which Niloy does not have).** `config_confrontation.yaml` and `NEW_STORY_TEMPLATE/config_newstory.yaml` were both pinned to `active_profile: lightning_l4`; switched both to `kaggle_2xt4` (num_gpus: 2, Q8, xformers, 1280×720, rate ₹0). Added a `KIT_PROFILE` env override in `src/cfgutil.py` so the hardware profile can be forced explicitly (prevents silent wrong-GPU runs). Rewrote `RUN_TUNED.sh` for Kaggle 2×T4 (sets `KIT_PROFILE=kaggle_2xt4`, removed the Lightning-only `sudo shutdown` billing step, replaced with a Kaggle 'Stop session' note). Verified no config points at L4

anymore; `cfgutil.py` passes `py_compile`. Reshipped `GODSHAND_MASTER.zip` (239 files).

- **25 Jul 2026, ~7:52 AM IST — Enforced "everything from the master folder."** Verified in code that every script roots to the master folder (`root = parent of src/`) and reads all inputs from there — story (`shots_file`), characters (`refs/`), workflows, grade targets. The only outside doorways are the *opt-in* `GODSHAND_ANCHORS` launcher copy (copies INTO `kit/refs/`) and `lipsync.py`'s lookup for the external LatentSync tool repo (not identity). Hardened `preflight.py` (already stage 1/10 of every run) to (a) print `MASTER FOLDER: <root>` and "all inputs read ONLY from this folder," and (b) warn about any stray ref image outside `face/ body/ costume/`. Reshipped `GODSHAND_MASTER.zip` (239 files); `py_compile` clean.
- **25 Jul 2026, ~7:48 AM IST** — Applied the **permanent code fix** for the Spider-Man bug and reshipped `GODSHAND_MASTER.zip` (238 files). Patched `comfy_client.py._first_image()` (the real render-conditioning leak — it fed the first image, which was the `_inbox` Spider-Man) and `render_motion.py._identity_score()` (the QC gate) to **skip any underscore-prefixed staging folder** (`_inbox`) while still reading `face/` and `body/`. Both files pass `py_compile`; behaviourally proven — with junk dropped back into `_inbox`, the loader now picks `body/jack_body_0.jpg` instead of `_inbox/spiderman.jpg`. Junk can no longer reach a render even if it reappears.
- **25 Jul 2026, ~7:25 AM IST** — Traced and documented the **Spider-Man/Joker root cause** in code: recursive ref glob (`refs/jack/**/*`) reaching into `_inbox/`, `_inbox` sorting first, `refs[:5]` keeping the strays, and the identity-check gate using the same contaminated glob so it approved the drift. Added the "why Spider-Man appeared" section plus the permanent one-line fix (glob `refs/<char>/face/*`, not the parent).
- **25 Jul 2026, ~7:22 AM IST** — Built the clean master kit `GODSHAND_MASTER.zip` (237 files) and delivered it to you as a download. Base = FINAL_ (correct full launcher, 10,565 B); added `RUN_TUNED.sh`; removed every `refs/*/_inbox/` folder (the stray Spider-Man / Joker / Scarlett images). Verified cast faces intact (jack 5, julie 5, kunal 7, observer 5) and `NEW_STORY_TEMPLATE/`, `l4_model_setup.py`, and the notebooks all present.
- **25 Jul 2026, ~7:15 AM IST** — Ran a full file-by-file audit (content hashes, nesting normalized). **Corrected** the earlier "nothing was lost" claim on this

page. Documented: `NEW_STORY_TEMPLATE/` dropped from TUNED_AK & FINAL_AK; launcher dropped from TUNED_AK and **regressed** in FINAL_AK; `l4_model_setup.py` and `notebooks/` dropped from TUNED_AK; `README_L4.md` dropped from TUNED_AK and regressed in FINAL_AK; `RUN_TUNED.sh` only in TUNED_AK. Changed the recommended master build from TUNED_AK → **FINAL_ (Downloads)**.

- **25 Jul 2026, ~7:00 AM IST** — Created this page as the durable build map after the four kit builds were first compared.

🔬 GODSHAND Engine Audit — Code + Real Run Evidence (25 Jul 2026)

📋 Session Log — 25 Jul 2026, Evening (End-to-End Audit Completion)

Creator Guide

The clearest creative-operational explanation of how the system turns story into output.

The following pages reproduce the original source PDF as part of this curated collection.

GODSHAND — Creator Guide (Story, Voice, Music & Multi-Character)

🎬 Two inputs only: a STORY_YAML block and one image per character. From those, the kit renders the video, voices the dialogue, and scores the music — no audio files needed.

What you provide vs. what the kit makes

You provide	The kit generates
STORY_YAML — story, shot list, spoken `line:` text, and `song:` mood labels (all text)	Video from character anchors + each shot's `action:` (Wan image-to-video)
One face image per character	Voice — Kokoro TTS reads the `line:` text you typed in the YAML
(nothing else)	Music — MusicGen composes a bed from the `song:` label, auto-ducked under the voice

The one seam: STORY_YAML

The setup cell has a single variable, `STORY_YAML`. On run it compiles:

```
STORY_YAML → _build_shots_from_story() → shots_confrontation.json → render loop
```

To change the film, you edit only this block. Nothing else.

Top-level keys

Key	What it is
`film`	title string
`note`	one-line logline
`look`	global style, applied to every shot unless a shot overrides it
`avoid`	global negative prompt
`characters`	map of name → `{ look, anchor }`
`shots`	the list — each entry is one render unit

Do not add a `render:` block — the validated Wan quality settings live in the notebook config, and overriding them risks the melt/rainbow regression.

Shot fields

Field	Meaning
`id`	unique string, e.g. `scene01_shot_03`
`secs`	duration in seconds (default 4)
`who`	list of character names (must match `characters` keys)
`camera`	camera move, e.g. "slow push-in", "low-angle crane up"
`action`	the main description: subject + emotion + camera, ending with the anti-melt tail
`line`	the spoken words (also accepts `say:` / `dialogue:`)
`talking` / `voiceover`	booleans
`song`	music mood label (shots sharing a label share one bed)
`palette`	color mood
`look`	per-shot style override (optional)
`takes`	optional, overrides rejection_samples for that shot

Every `action:` must end with the anti-melt tail — it is what holds the image together: `smooth steady camera movement, gentle natural motion, subtle drifting atmosphere and soft shifting light, stable coherent shapes, solid consistent forms, a living breathing cinematic shot, no freeze`

Generating a story with an LLM (Grok)

Paste your prose story into this prompt and drop the YAML it returns into `STORY_YAML`:

```
You are a shot-list compiler for an image-to-video AI film kit.
I will give you a story in prose. Convert it into YAML in EXACTLY this schema.

Rules:
- Output ONLY valid YAML, no commentary, no code fences.
- Top-level keys: film, note, look, avoid, characters, shots.
- characters: give each person a name, a `look:` description, and an `anchor:`
  filename like name_anchor.png. Reuse jack (man_anchor.png) and julie
  (woman_anchor.png) for the two shipped faces.
- Every shot needs a UNIQUE id like scene01_shot_01.
- secs: 4 (keep all shots 4 seconds).
- who: list only character names defined above.
- Put spoken dialogue in `line:` and set talking: true (or voiceover: true).
- Give each shot a `song:` mood label; reuse the same label across a scene.
- action: describe the moment cinematically (subject + emotion + camera move),
  then ALWAYS end the action with this exact tail:
  "smooth steady camera movement, gentle natural motion, subtle drifting
  atmosphere and soft shifting light, stable coherent shapes, solid consistent
  forms, a living breathing cinematic shot, no freeze"
- Do NOT include a `render:` block. Do NOT change resolution/steps/model.

Here is the story:
<PASTE YOUR STORY HERE>
```

Voice + music

- Voice — Kokoro TTS (English), set up in the audio step. It reads the `line:` text from each shot. Voice is synthetic narration/dialogue, not a specific person's cloned voice.
- Music — MusicGen (`facebook/musicgen-medium`), keyed by the per-shot `song:` label. Shots sharing a label get one continuous bed. It is a generated instrumental score, not a licensed track.
- Mixing — `stitch.py` ducks music to −4dB under the voice and fixes the stereo layout automatically.
- Tip — for a clean demo, use one or two `song:` labels for the whole piece rather than a new one per shot.

Multi-character support (patched notebook)

The patched notebook (`rainman-wan-multichar.ipynb`) adds two things:

1. N-character ingestion — after the jack/julie bind, it loops over every character in `STORY_YAML` that declares an `anchor:`, finds that image anywhere under `/kaggle/input`, and installs it to `refs/<name>/face/` so it is identity-locked. Characters with no matching image fall back to text-forge (unchanged behavior). The original 2-character path is untouched.
2. Spoken-line fields — every shot now carries `line` / `vo_text` into `shots_confrontation.json`.

To add a character: give it a name in `characters:` with a `look:` and an `anchor:` filename, then upload an image with that exact filename (e.g. `elder_anchor.png`) to the Kaggle dataset. No image = text-forged from the look tokens.

Example: THE LANTERN KEEPER (runs as-is)

Validated against the real patched builder — 3 characters, dialogue, and music all parse through:

```
film: THE LANTERN KEEPER
note: Demo - multi-character story with voice and music. Runs
on the kit as-is.

look: warm analog neon twilight, soft synth-wave glow, magent
a and amber haze, gentle grain, nostalgic hush, 35mm
avoid: warped faces, extra fingers, duplicate limbs, morphin
g, flicker, text, watermark, jitter, plastic skin

characters:
  jack:
    anchor: man_anchor.png
    look: a weary man in his forties, olive field jacket over
a white tee, short dark hair, tired honest eyes, faint stubbl
e
  julie:
```

```
    anchor: woman_anchor.png
    look: a calm luminous woman, soft dark hair, gentle stead
y gaze, pale scarf, quiet warmth like an angel in human form
  elder:
    anchor: elder_anchor.png
    look: an old lantern keeper, deep-lined kind face, silver
beard, worn brown coat, holding a small brass lantern, patien
t eyes

shots:
- id: lantern_01
  secs: 4
  who: [jack]
  camera: slow push-in
  song: prelude
  talking: true
  line: I kept the light because someone had to.
  palette: amber lamp glow
  action: a slow push-in on the weary man standing in a dim d
oorway holding a small brass lantern, speaking softly to hims
elf, warm lamplight on his face, smooth steady camera movemen
t, gentle natural motion, subtle drifting atmosphere and soft
shifting light, stable coherent shapes, solid consistent form
s, a living breathing cinematic shot, no freeze

- id: lantern_02
  secs: 4
  who: [jack, julie]
  camera: slow orbit
  song: prelude
  talking: true
  line: You never had to carry it alone.
  palette: magenta and amber haze
  action: a slow orbit around the man and the luminous woman
facing each other in the neon haze as she reaches toward the
lantern between them, tender recognition, smooth steady camer
```

```
a movement, gentle natural motion, subtle drifting atmosphere
and soft shifting light, stable coherent shapes, solid consis
tent forms, a living breathing cinematic shot, no freeze

- id: lantern_03
  secs: 4
  who: [elder]
  camera: low-angle crane up
  song: hymn
  voiceover: true
  talking: false
  line: Every keeper passes the flame; none of us owns it.
  palette: quiet neon fade
  action: a low-angle crane up on the old lantern keeper rais
ing his brass lantern into the twilight as embers drift upwar
d, quiet reverence, smooth steady camera movement, gentle nat
ural motion, subtle drifting atmosphere and soft shifting lig
ht, stable coherent shapes, solid consistent forms, a living
breathing cinematic shot, no freeze
```

Creator workflow

1. Write or generate the story (use the Grok prompt above), defining each character in `characters:`.
2. Upload one face image per character to the Kaggle dataset, named to match each `anchor:` (jack/julie use `man_anchor.png` / `woman_anchor.png`).
3. Paste the YAML into `STORY_YAML`. Add `line:` + `talking`/`voiceover` for speech and a `song:` label for mood.
4. Run the setup cell — the kit compiles the shots and reports which characters are identity-locked vs text-forged.
5. Render a short chunk (`KIT_CHUNK_SIZE` small), inspect frames, then scale.
6. Package — the stitched clip + zip drop out automatically.

Honest limits (don't oversell)

- Voice is synthetic TTS (Kokoro, English) — not a specific person's cloned voice unless voice-cloning is wired later.
- Music is a generated instrumental bed — mood-matched, not a bespoke human score or licensed song.
- Lip-sync is approximate — `talking: true` drives mouth motion but is not frame-perfect to the words. Good for cinematic VO/ambient dialogue, not a talking-head substitute.
- Voice last-mile — the `line:` text is captured into the shots JSON, but Kokoro actually speaking it depends on one wire in `src/render_motion.py`, confirmed on the next live run. Music and video work today.
- Multi-character-in-one-frame is the hard case for any image-to-video engine — identity holds strongest for the character whose anchor seeds a shot.

For demo recording

Show the loop small: 2–3 characters, 2–3 shots, one `song:` label, one spoken `line:`. That demonstrates the three selling points live — your own cast, your own story, voice + music baked in — without a long render wait. Say the "scales to a full cast and feature length" claim; show the short multi-character clip.

Demo Film: The Beholder's Truce

The proof-of-concept film brief that explains the emotional sales logic of the kit.

The following pages reproduce the original source PDF as part of this curated collection.

GODSHAND Demo Film — “THE BEHOLDER'S TRUCE” (micro-memoir, ~30 min render)

Your first job after the current run finishes. A tiny, self-contained proof film built from the anchors you already have — 3–4 shots, ~30 minutes of render. This becomes the clip that sells the kit and headlines every post. No new film. Just this.

Logline

At the end of a long road, a weary man of forty-seven meets the angel who has walked every mile beside him — and finds his truce. A wordless 30-second memoir, distilled from *The Saga of the Beholder* (Chapter V — The Beholder's Truce).

Why this works as a demo

- **It's short enough to actually finish** — 3–4 shots, one render session.
- **It proves the hard stuff in 30 seconds:** a consistent human face, a second consistent character, a real film grade, and an *emotional hold* (not just a moving filter).
- **It's emotional, not technical** — buyers feel it before they analyze it. That feeling is what sells.
- **It uses only what you already have:** `man_anchor.png` (the man) and `woman_anchor.png` (the angel), current engine, current grade.

The film (shot list)

#	Anchor	What we see	Motion	~Len	What it shows off
1	Man	Wide: a lone figure on an empty desert road at dusk, long shadow, drifting dust	Slow push-in, heat shimmer	4s	Environment + motion + warm Kodachrome grade

#	Anchor	What we see	Motion	~Len	What it shows off
2	Man	Extreme close-up: the weathered 47-year-old face, dust on skin, a slow blink and breath	Micro head-turn, eyes settle toward camera	5s	**Face consistency + skin detail + film grain** — the shot that silences AI skeptics
3	Angel	Backlit figure of the angel in soft haze, light blooming around her	Gentle drift forward, hair/cloth motion	4s	**Second consistent character** • halation/bloom + ethereal grade
4	Man	The man lifts his eyes; a faint exhale of acceptance as warm light breaks over him	Slow tilt up, lighting shift	5s	**Emotional hold + a real lighting change** (proves it's rendered, not filtered)
5 *(optional)*	Man	Final wide: he walks on, smaller in frame; title fades in	Locked / slow drift	4s	Closes the arc; carries the title card

Grade & sound

- **Grade:** your current look — Kodachrome warmth, halation on the highlights, fine film grain, slightly desaturated shadows. Don't change the pipeline; the demo should look exactly like what buyers will get.
- **Sound:** silence + wind, and a single held piano note under shots 3–4. Let the image breathe.
- **Title card:** *THE BEHOLDER'S TRUCE* — then small: *rendered entirely on a free GPU.*

Render budget

- Core film = **4 shots × ~8–9 min ≈ 32 min**. That's your ~30-minute target.
- Shot 5 is optional (+~8 min) if quota allows.

- Edit + grade assembly afterward is minutes, not hours — the grade is already baked per shot.

How it plugs into the current setup

- Uses the existing anchors and the notebook you already have (streaming grade fix baked in).
- Render each shot as its own short clip, then stitch in any editor (or the kit's stitch step).
- The moment it's exported: it's your **proof asset** (listing) *and* the raw material for the walkthrough's before/after and the 60–90s sales cut.

Kit Listing and Setup Sheet

Commercial-facing packaging copy and setup positioning for the offer.

The following pages reproduce the original source PDF as part of this curated collection.

GODSHAND — Cinematic AI Film Kit (₹499 Listing + Setup Sheet)

Turn a single portrait into a cinematic AI film shot — on a free GPU. GODSHAND is the exact pipeline used to render *The Saga of the Beholder*. You bring a character image; the kit gives you graded, HD, motion-consistent film frames without a render farm.

> Draft listing copy — rename "GODSHAND" freely. Everything below is written to be pasted into Topmate / Razorpay / Gumroad. The second half is the honesty sheet buyers should read *before* paying.

The one-liner

GODSHAND Cinematic AI Film Kit — a guided notebook that renders film-grade AI video shots from your own character images, running entirely on free Kaggle dual-T4 GPUs. No render farm, no paid cloud, no melt.

What it does

- Takes your **anchor image(s)** (a face/character) and animates them into short cinematic shots.
- Runs a hardened **Wan 2.2 image-to-video** pipeline (with a working LTXV fallback engine).
- Applies a full **film grade** — Kodachrome-style color, halation, film grain, sharpening — automatically.
- Upscales to **HD (1536×1024)** and interpolates for smooth motion.

- Renders in **shot batches** so you can build a whole film across multiple free-GPU sessions.

Why it's different (the hard-won part)

This isn't a fresh ComfyUI workflow you have to debug. It's a kit where the painful failures are **already solved**:

- ✅ **Anti-melt / anti-rainbow fix** — the render-schedule desync that turns AI video into noise soup is patched.
- ✅ **Memory-safe streaming grade** — the postprocess step no longer OOM-crashes on HD footage (bounded RAM, tunable).
- ✅ **Fits free 14 GB T4 GPUs** — quantized models chosen so it actually runs where you don't pay.
- ✅ **Shot/chunk system** — render 435-shot films in resumable batches.

What's inside

- The full `rainman-wan.ipynb` Kaggle notebook (drop-in, guided).
- The GODSHAND source kit (render, grade, upscale, face-enhance, stitch).
- A quick-start guide + the recommended settings that actually work.
- **A full video walkthrough** — the entire process screen-recorded end to end, so nothing is left to guesswork.
- Sample anchor images and a demo shot list.

Proof

The film speaks for the kit. *The Saga of the Beholder* — every frame rendered on free GPUs with this exact pipeline. **The demo clips are the use-case: film entertains, the kit empowers.**

Price

₹499 — one-time, lifetime access. Includes the full kit, the notebook, the quick-start guide, **and a complete step-by-step video walkthrough** of the entire

process — recorded end to end so you just follow along, click for click.

⚠️ Read before you buy — honest setup requirements

> 🧭 This is a **guided notebook**, not a one-click app. If you follow steps and have a little patience, it works. If you expect a magic "make my movie" button, this isn't that — and I'd rather you know now.

You will need

- [ ] A free **Kaggle account**, phone-verified (required to unlock GPUs).
- [ ] Kaggle's free **dual-T4 GPU** quota (~30 hrs/week — enough for real work, not unlimited).
- [ ] A free **HuggingFace account + access token** (to download the open models).
- [ ] Your own **character/anchor images** (or use the samples included).
- [ ] Basic comfort with: opening a notebook, pasting a token, clicking Run. **No coding required.**

Time & expectations

- Rendering is **slow on free GPUs** — roughly **8–9 minutes per shot**. A full film is many sessions, not one afternoon.
- Output is **stylized AI video**. Expect occasional imperfections (hands, tiny details, rare artifacts). The kit minimizes them; it doesn't make them impossible.

What this kit does NOT do

- ❌ It's not a hosted service — you run it on your own free Kaggle account.
- ❌ It won't run well on random low-VRAM setups; it's tuned for **Kaggle T4s**.
- ❌ It doesn't include a commercial-use guarantee for the underlying open models — **check each model's license** if you plan to sell what you render.

- ❌ It's not a chatbot or a script writer — it renders shots from images you provide.

FAQ

Do I need a registered business or GST to buy?

No — it's a digital download. You just need a way to pay.

Can I use my own faces/characters?

Yes. Drop in your own anchor images. Sample images are included to start.

Will it work on Google Colab?

It's built and tested for Kaggle dual-T4. Colab may work with tweaks but isn't supported out of the box.

Can I sell videos I make with it?

The kit code is yours to use, but the underlying AI models have their own licenses. Verify commercial terms for the models before selling output.

What support do I get?

Best-effort help via the setup guide + a contact channel. This is a solo-built kit, not a company helpdesk — priced accordingly.

Refund policy (suggested)

Because this is an instant digital download **with a complete video walkthrough included**, all sales are final once the files are accessed. There's no "I couldn't set it up" gap — the video shows every single step. If you still get stuck, reach out and I'll help you get running.

Pre-launch checklist

- [ ] Finish + export the film (your #1 proof asset).
- [ ] Pick a payment rail (Topmate / Razorpay individual / Gumroad).
- [ ] Record a 60–90s screen demo of the kit rendering one shot.
- [ ] Zip the kit + guide; test the download on a clean account.

- [ ] Publish listing → share proof clips → get first paying stranger.

🎥 Video walkthrough — recording script

> 🎬 Record on a **clean, fresh Kaggle account** (new login, no pre-setup). The buyer must see the exact path they'll walk — not your already-configured machine. Timelapse the slow render parts. Aim for **12–18 min** with chapter markers.

0. Cold open (10–20s) — the hook

Open on a finished graded shot from the film. Voiceover: "This was rendered on a *free* GPU, no paid cloud. In the next few minutes I'll show you exactly how — step by step, nothing skipped."

1. What you need (30s)

On screen: a browser, a free Kaggle account, a free HuggingFace account. "No installs. No coding. If you can paste a code and click Run, you can do this."

2. Kaggle account + GPU unlock (1–2 min)

Sign in → phone-verify → show where to enable the **dual-T4 GPU** accelerator. Call out the ~30 hrs/week free quota honestly.

3. HuggingFace token (1 min)

Create account → Settings → Access Tokens → create a read token → copy it. "This lets the notebook download the models."

4. Load the notebook + kit (1–2 min)

Import `rainman-wan.ipynb` → attach the GODSHAND kit dataset → add the token as a **Kaggle Secret** named `HF_TOKEN`. Show exactly where each field goes.

5. Add your character + story (2–3 min) — the magic moment

Upload your **anchor image(s)** → open the shot-list / story file → edit one shot's prompt so viewers see *their* story is what drives the render. This is the "it's *my* film" hook.

6. Run it (1 min + timelapse)

Click Run All. Narrate what's happening: models downloading, both experts loading, sampling starting. Timelapse the wait so it never feels slow.

7. First shot renders (1–2 min)

Show the raw base shot. Then the **HD upscale** and the **film grade** — cut a quick before/after so the color + grain "wow" lands.

8. Export + download (1 min)

Show the final `.mp4` and where to download it. "That's one shot. Repeat for your whole film — the kit resumes across sessions."

9. Outro (30s)

Recap the 3 things they needed. Point to the quick-start guide + how to get help. End on the finished film clip again. "That's the whole thing. Go make yours."

Recording tips

- Screen-record at 1080p; a small webcam bubble builds trust but is optional.
- Cut dead air ruthlessly; every model-download wait gets timelapsed.
- Add chapter markers matching the steps above — buyers rewatch specific parts.
- A 60–90s cut of steps 0, 5, 7 = your **sales demo**. Film both in one sitting.

🔴 How to record it (free)

Screen recorder: OBS Studio (free, all platforms) — or zero-install built-ins: Windows **Win+G**, Mac **Cmd+Shift+5**. Record at 1080p.

Don't film the slow render live. Record the setup steps → **stop** during the ~8–9 min render → **resume** for the result. Splice the segments (or drop a "rendering..." timelapse card) in editing.

Audio: narrate live with any headset/phone mic, *or* record silent and add captions + voiceover after — less pressure on take one.

Editing (free): CapCut, DaVinci Resolve, or iMovie / Clipchamp. **Import the actual film `.mp4` clips** for beats 0 and 7 — don't screen-record them, or they'll look soft.

Before you hit record: use a **clean browser profile** (bookmarks hidden, no personal tabs) and the **fresh Kaggle account**, so viewers see the real from-

scratch path.

🎯 Where to find buyers

🧭 Honest targeting note: the kit still needs a Kaggle account + HF token, so the real sweet spot isn't *literally* zero-tech — it's the **budget AI-creator who'll happily follow a video**. The faceless-YouTube / AI-video crowd is perfect: hungry, tutorial-native, allergic to paying for expensive tools.

💰 Demand is real: creators already pay for faceless-video workflows (one builder even got an $8K offer to buy their pipeline). Your edge is a one-time ₹499 kit that runs free on Kaggle — it directly beats the monthly SaaS tools this crowd resents.

Communities to target

Platform	Community	Why / rules to respect
Discord	Faceless Incubator (~38k)	Largest faceless-creator hub — biggest pool. Watch promo rules.
Discord	YouTube Faceless (~20.7k)	The #1 faceless community, but **strictly no self-promo** — value first, DM only if invited.
Discord	AI Forum for India (~7.5k)	India-focused AI hub — your budget-creator base.
Discord	AI Filmmakers server (invite pinned in r/aifilmmaking)	Craft-focused, platform-independent; share the film as proof.
Discord	Wan (official) / ComfyUI servers	The exact models your kit uses; showcase channels welcome results.
Reddit	r/aitubers	Small AI-video creators, tutorial-hungry. Showcase-friendly.
Reddit	r/aifilmmaking	Exactly your craft; people asking “how was this made?”

Platform	Community	Why / rules to respect
Reddit	r/comfyui, r/StableDiffusion	Technical AI-video crowd; best for "built on free GPUs" proof clips. No paywalled-workflow spam.
Reddit	r/FacelessAICreators, r/Seedance_AI	More faceless / AI-video subs; showcase-friendly.
India	AI Learners India (Hindi, no-code Discord)	Zero-tech-background, limited-funds beginners — your ideal buyer.
India	AI Video Generators (Telegram)	Active AI-video channel comparing tools; warm audience.
Facebook	"Faceless YouTube" creator groups	Older/less-technical creators on tight budgets.

How to find more (server-finder sites)

Search these by tag (`comfyui`, `filmmaking`, `ai-video`, `faceless`) to surface fresh servers: **Disboard**, **Discadia**, **top.gg**, **DigitalOcean's AI-Discord roundup**, and **aixploria's AI-Discord list**.

The value-first playbook (don't get banned)

1. **Lead with the film, not the link.** Post a stunning clip: "Rendered this entirely on a free Kaggle GPU." Let people ask *how*.
2. **Answer generously in public.** When someone asks about setup, actually help. You become "the free-GPU AI-film person."
3. **Soft-mention, don't spam.** "I packaged my exact pipeline + a full walkthrough video if anyone wants the shortcut — happy to DM."
4. **Use showcase channels** where self-promo *is* allowed; respect the no-promo rule everywhere else.
5. **Let the walkthrough clip do the selling** — people who see the whole process work are pre-sold and won't refund.

Ready-to-post templates

Reddit showcase post (r/aifilmmaking, r/StableDiffusion, r/aitubers)

> **Title:** I rendered a short film entirely on a *free* GPU — no paid cloud, no render farm

> Body: I spent months fighting the usual AI-video curse — melting faces, rainbow noise, characters that change identity every frame. I finally got a pipeline that holds a consistent face *and* grades like real film (Kodachrome warmth, grain, halation), running entirely on free Kaggle T4 GPUs.
> Here's a ~30-second piece from a longer memoir project I'm building.
> Ask me anything about how it's built — happy to share the setup. What would you want to see it render next?

(Attach the clip. No link in the post body — let people ask. Reply to "how?" with the soft-mention below.)

Discord showcase-channel message

> Made this on a **free** Kaggle GPU 👇 Took forever to kill the melt + rainbow noise, but it finally holds a real face and a proper film grade. Everything's open models on a free T4 — happy to answer setup questions 🙌

(Drop the clip. Only post in channels that allow showcases.)

Soft-mention reply (when someone asks "how did you do this?")

> Sure — it's Wan 2.2 image-to-video on ComfyUI, quantized to fit a free T4, with a custom grade + upscale pass. I packaged my exact notebook + a full step-by-step walkthrough video so people don't have to relive the debugging pain — happy to DM the link if that's useful, otherwise glad to explain any part here.

Project Tutorials

Step-by-step material that makes the kit and related bundle easier to follow end to end.

The following pages reproduce the original source PDF as part of this curated collection.

📘 Project Tutorials — Step by Step (All Projects + Kit)

📘 Your complete, follow-along setup guide for all four products — the **GODSHAND film kit** plus the three software projects (**Convergence**, **SafeScreen**, **Solana Sniper**). Each section goes top to bottom, click by click, so you — or a buyer — never have to guess.

> **Note on the Google Drive link:** I couldn't open the Drive folder you shared — Drive folder links load their file list in the browser and can't be read from a share URL, and I don't have a Drive connector. This guide is built from your own project pages in this workspace instead, which are accurate and current. If you want any project's *exact* file names/folder layout baked in, paste that project's file list here (or drop the project's `.zip` into our chat) and I'll fold it in.

How to use this guide

- Each project is **self-contained** — jump straight to the one you need.
- **What you need** lists prerequisites — get those first.
- Steps are in order. Don't skip; each unlocks the next.
- **Honest limits** is what the product does *not* do. Keep it in every listing — it's your refund shield.

---

1) 🎬 GODSHAND — Cinematic AI Film Kit

Open the listing + setup sheet

What it is: turns your character image into graded, HD, motion-consistent film shots — running on a free Kaggle GPU. This is the exact pipeline we set up

together for the 2-minute cut of *The Saga of the Beholder*.

What you need

- [ ] A free **Kaggle** account, phone-verified (unlocks GPUs)
- [ ] A free **HuggingFace** account + a read **access token** (optional, but gives faster model downloads)
- [ ] The **GODSHAND kit** uploaded as a Kaggle **Dataset** — it must contain a `src/` folder with `render_motion.py` inside
- [ ] Your **character/anchor images** as a Kaggle Dataset (or use the samples)

Step by step

1. **Upload the kit as a Kaggle Dataset.** Take the kit ZIP (`GODSHAND_KIT_AUTOPILOT.zip`) → Kaggle → *Datasets* → *New Dataset* → upload. Kaggle auto-extracts it, so the `src/` folder is detected automatically.
2. **Create/import the notebook.** New Notebook → *File* → *Import Notebook* → upload `rainman-wan-multichar.ipynb` (the version with the 2-minute cut).
3. **Attach your datasets.** In the notebook's right sidebar: **Add Input** → **Datasets** → add (a) your GODSHAND kit dataset and (b) your `saga-custom-characters` refs dataset.
4. **(Optional) Add your HF token.** *Add-ons* → *Secrets* → new secret named `HF_TOKEN` → paste your token. Skipping this still works; downloads are just slower.
5. **Set the machine.** Right sidebar: **Accelerator = GPU T4 × 2**, **Internet = On**.
6. **Run All.**
7. **Confirm the setup log.** You should see `GODSHAND kit mounted…` and then `AUTO-DERIVED from story: total=28 shots` — that's the 2-minute cut building correctly.
8. **Let it render.** It renders shot by shot and skips any already finished.

> ⚠️ **It runs on ONE T4, even on the "T4 × 2" machine.** A built-in RAM guard only ever launches one render server (two would exceed the 32 GB host

RAM and crash). So shots render **one at a time**, not in parallel — this is expected, not a bug.

The 2-minute cut (already configured for you)

- **28 shots, ~1:57** — the five opening scenes (`i01_jungle → i05_between`).
- `rejection_samples = 1` — one take per shot (fastest).
- **Net render time: ~4.5–5.5 hours** on a single T4 (~8–9 min/shot).

If a session stops before finishing (resume)

With the free **~2 GPU-hours/week**, it won't finish in one go — it stops cleanly at the budget and saves `Beholder_progress.zip` + a `STATUS.txt` listing what's left.

1. Next session: **Add Input** → **Your Work** → **this notebook** → **its Output**.
2. **Run All** again — it detects finished clips and picks up where it stopped.
3. When all 28 are done, it auto-stitches the final master into the zip.

Honest limits

- Guided notebook, **not one-click**; needs a Kaggle account (+ optional HF token).
- Single-T4 sequential rendering; ~8–9 min/shot.
- Stylized AI output — occasional artifacts (hands, tiny details).
- Check each underlying model's **license** before selling what you render.

---

2) 🎯 Convergence Framework — ₹499

Open the listing

What it is: a thinking framework + ready-made LLM prompts to define the *right* objective before you build — so you can decide in an hour whether a goal is even formalizable.

What you need

- The kit ZIP (a PDF + a few markdown files). **No install, no GPU.**
- Any LLM (ChatGPT / Claude / Gemini).

How to use it

1. Open `Convergence_Under_Constraint.pdf` — read the core method first.
2. Follow `README_objective_function.md` — walk Steps 0–8 to write your objective function.
3. Paste `SystemPrompt_Code_Convergence_Deterministic.md` into your LLM as a *system prompt* before coding.
4. Use `AI_Code_Review_Prompt_v2.md` to review code against the objective (not just syntax).
5. Study `Trading_Bot_Convergence_Framework_v3.md` as a fully worked example, then copy the pattern for your own system.

To make it demo-ready (before selling)

- Apply it to **3–4 real goals**, including one deliberately messy / non-formalizable one, and show it correctly flags the messy one.
- Record a short worked example: *real goal → run the framework → verdict.*

Honest limits

- It's a **thinking discipline, not runnable code** — the value is the judgment, not an executable.

---

3) 🛡️ SafeScreen — ₹1,999

Open the listing

What it is: a trained on-device content-moderation model + inference SDK — flags/masks violence, gore, and NSFW in images, with an OCR layer for text. Runs offline, no cloud.

> ⚠️ **Redact secrets FIRST.** `nsfw_fix.py` contains a hardcoded Kaggle username + API key. Delete those lines and **rotate the key** before you

zip or share anything.

What you need

- Python + a fresh virtual environment. Runs on **CPU** (onnxruntime) — no GPU required.

How to run

1. Create a fresh venv and install the project's dependencies.
2. **Sanity check first:** confirm `toxic_detector.onnx` loads and classifies **one** image end-to-end with onnxruntime on CPU.
3. Run the inference scripts: `inference_onnx`, `inference_tflite`, or `batch_inference`.
4. Build a **held-out test set you did NOT train on** → report **precision, recall, F1, a confusion matrix, and false-positive rate — per class**.
5. **Live demo:** a minimal Gradio or FastAPI page — upload an image, get the flagged label + confidence, fully offline. Screen-record it.

Honest limits

- Per-class accuracy varies — only market the classes where recall is genuinely strong; flag the weak ones. Buyers must test on their own data.

---

4) 📈 Solana Sniper — ₹2,499

Open the listing

What it is: the full source + a rigorous **6-month paper-trading study** of a Solana memecoin sniper (ingestion, ML signal layer, risk manager, React/TS dashboard, Monte Carlo risk analysis). Sold as a **study**, not a live money-maker.

> ⚠️ **Redact secrets FIRST.** Delete `.env`, `.env1`, and `gmgn_keys/gmgn_private.pem`; **rotate** the wallet + API keys (assume burned); add a `.gitignore` and a clean `.env.example`.

How to run (paper mode only)

1. **Hard safety rule:** never connect real funds, real keys, or mainnet execution. Paper / backtest only.
2. Set up a fresh environment and install dependencies.
3. Re-run the **Monte Carlo / backtest harness** on historical data.
4. Report the **equity curve, win rate, and max drawdown — losing runs included**, honestly.
5. Reproduce the study's output from the shipped ZIP with a **single command**, and record it.

Honest limits

- Paper-tested only; live performance unverified. **Not financial advice; past results don't predict future results.**

🧰 Handy notes for all four

- **Free tool to run/fix code:** Antigravity (free agentic IDE) for the CPU projects; fall back to Cursor (free) or aider if it throttles. GPU work (the film) stays on Kaggle.
- **Always strip secrets** (`.env`, `.pem`, keys) and add a `.env.example` before zipping any project.
- **Record proof on a clean machine/account** so a buyer's first run matches your demo.
- **Fastest first rupee:** publish **Convergence (₹499)** first — it's clean, needs zero redaction, and is the shortest path to a sale.